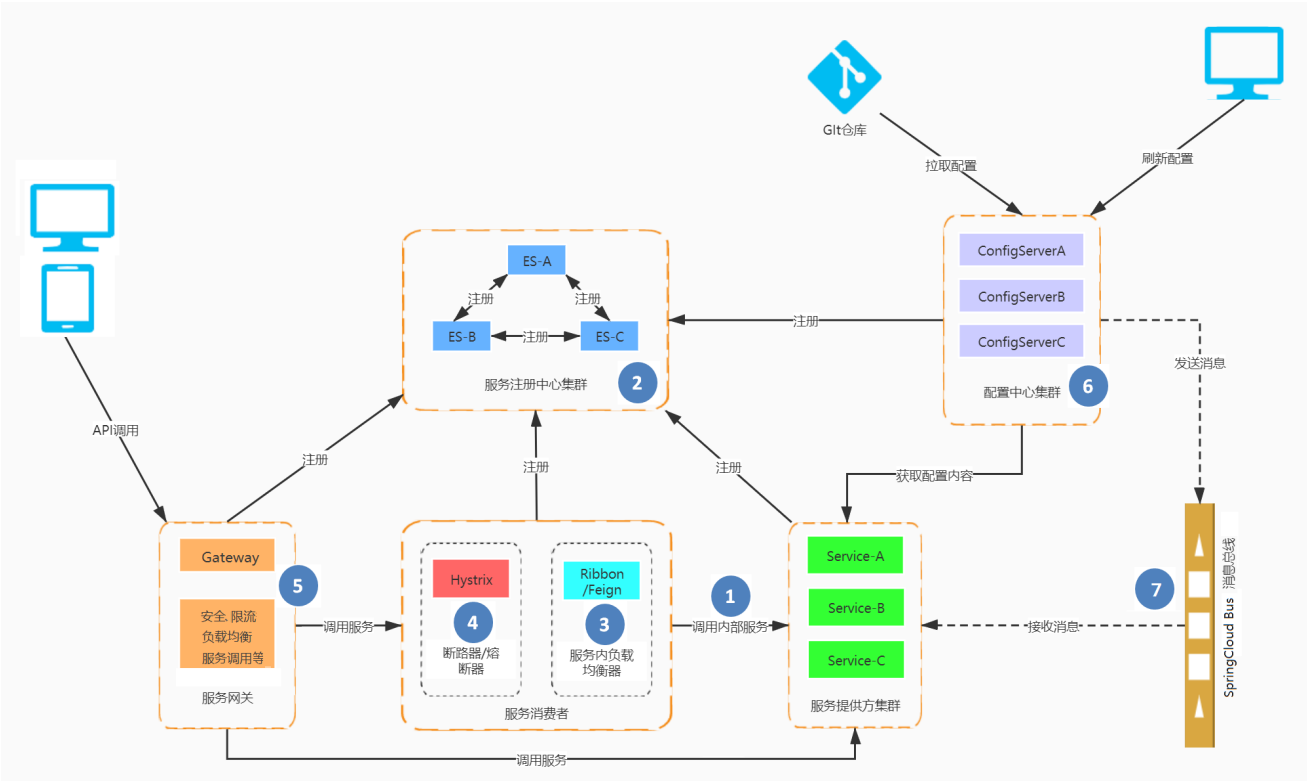
本章所讲内容为 Spring Cloud 架构图中的第七个部分。



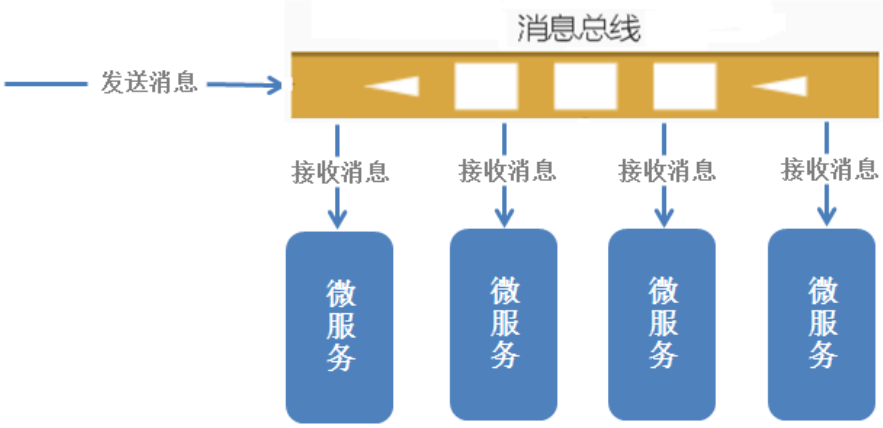
1.动态刷新配置

微服务项目中，使用到Spring Cloud Config来托管各模块的配置文件后，会有一个尴尬的问题： Config服务端并不能动态的感知Git上配置文件的变化，当git上配置文件更新后，如果不采取其它措施，就只能重启相关应用，从而达到配置的更新。那么如何不重启项目， 就能实现批量刷新配置呢?

使用Spring Cloud Bus（消息总线）可以解决这一问题。它用于在集群中传播状态变化，可与Spring Cloud Config联合实现热部署。

1.1.消息总线简介

简单理解，消息总线就是一个消息中心，众多微服务实例可以连接到总线上，实例可以往消息中心发送或接收信息（通过监听）。这样，消息总线就充当一个中间者的角色，使得微服务之间的消息通信实现解耦。



Spring Cloud Bus 是 Spring Cloud 体系内的消息总线。

具体来说：在微服务系统架构中，通常会使用轻量级的消息队列（RibbitMQ、Kafka）来构建一个公用的消息主题（默认就是Spring Cloud Bus），并让系统中所有的微服务都连接到该主题上。这样，由该主题产生的消息，就会被所有微服务所监听到并消费，所以称Spring Cloud Bus为消息总线。

1.2.刷新配置原理

下面分析如何基于SpringCloud Bus实现SpringCloud Config的配置刷新。

1.2.1.指定刷新范围

SpringCloud Bus会对外提供两个HTTP请求接口（必须为post方式请求）：

- /bus/env：针对单个微服务实例修改或刷新其配置信息。
- /bus/refresh：借助于Spring Cloud Bus的消息机制，针对分布式系统中的所有微服务实例修改或刷新其配置信息。

这里我们使用/bus/refresh接口，对所有微服务实例进行配置刷新。

> 实际上，/bus/refresh接口也可以针对某个服务实例进行配置刷新。该接口提供了destination参数，用来定位具体要刷新的服务。比如：/bus/refresh?destination=customer:9000

1.2.2.指定刷新事件触发者

SpringCloud Config 配置刷新的触发有两种方式：

1. 当git仓库有更新时，自动调用/bus/refresh接口进行配置刷新。也就是说：由git自动触发刷新事件。
2. 手工调用/bus/refresh接口进行配置刷新。也就是说：由更新者主动触发刷新事件。

这两种方案的区别仅在于是不同的人触发了刷新接口。实际上，一般很少采用自动刷新的机制，都会在修改后，确认无误后再执行刷新。

1.2.3.指定全局广播模式

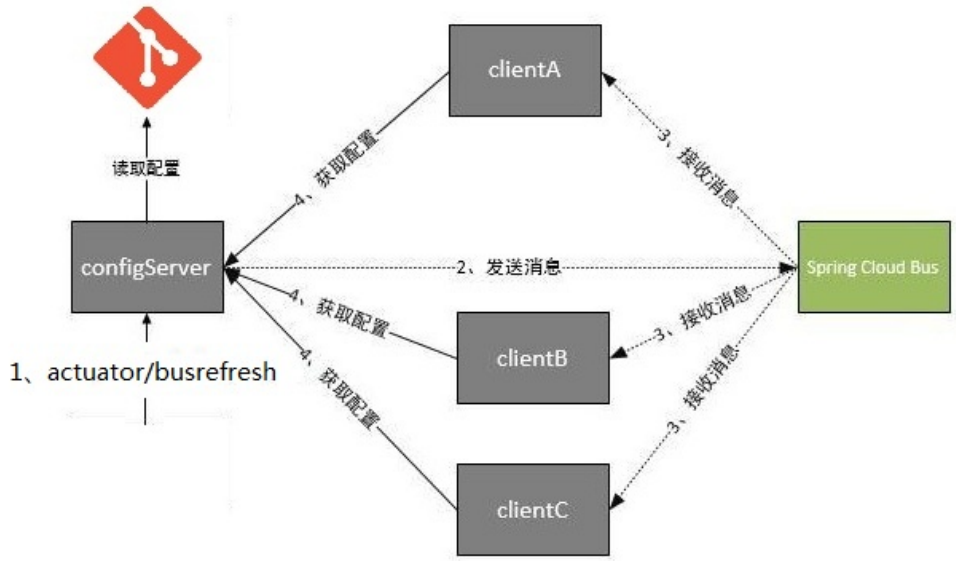
Spring Cloud Bus 的全局广播有两种设计思想：

- 利用消息总线触发某一个微服务的 /bus/refresh，进而刷新所有微服务的配置。
- 利用消息总线触发一个ConfigServer 的 /bus/refresh，进而刷新所有客户端的配置

我们采用第二种，第一种方式不适合的原因有：

1. 打破了微服务的职责单一性。负责业务模块的微服务不应该承担配置刷新的职责。
2. 破坏了微服务各节点的对等性。
3. 有一定的局限性。例如微服务迁移时，它的网络地址常常发生变化，如果想要做到自动刷新，还需要增加更多的配置。

1.2.4.刷新配置流程



根据上图可以看出利用Spring Cloud Bus做配置更新的步骤:

1. 将Config Server、微服务集群都引入Bus，也就是将他们都连接到了消息总线上。
2. 使用post请求给ConfigServer发送bus/refresh刷新请求。
3. ConfigServer接收到请求后从Git中更新配置并且发送给Spring Cloud Bus。
4. Spring Cloud bus接到消息后，利用RibbitMQ以广播的方式通知给其它微服务。
5. 其它微服务接收到通知，请求ConfigServer获取最新配置。
6. 全部微服务均获取到最新的配置。

2.动态刷新配置实例

下面通过使用Spring Cloud Bus与Spring Cloud Config的整合，并以RabbitMQ作为消息队列，实现应用配置的动态更新。

2.1.安装rabbitMQ

2.1.1.安装Erlang语言环境

1. 安装Erlang语言环境：otp_win64_xx.exe；
2. 安装后要配置环境变量：
   - ERLANG_HOME：otp安装路径
   - PATH：otp安装路径/bin
3. 测试是否安装成功。命令行中输入如下命令：

```
C:\Users\Administrator>erl
Eshell V9.3  (abort with ^G)
```

2.1.2.安装rabbitMQ

1. 安装rabbitMQ 产品：rabbitmq-server-x.x.x.exe
2. 进入rabbitMQ的sbin目录，安装插件

```
C:\Program Files\RabbitMQ Server\rabbitmq_server-3.7.4\sbin>rabbitmq-plugins enable
rabbitmq_management
```

3. 插件安装完成后，在window服务管理中重启服务。
4. 打开地址： http://127.0.0.1:15672/ 看到管理界面。用户名和密码都是guest

2.2.config server端配置

1. 修改 config_server_15000工程和config_server_15001 工程，添加依赖：

```
<!-- 消息总线依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
```

```
    <artifactId>spring-cloud-bus</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
</dependency>

<!-- 监听依赖 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. 修改配置文件：

```
server:
    port: 15000

spring:
    application:
        name: config-server
    cloud:
        config:
            server:
                git:
                    uri: https://gitee.com/yuhongjun01/scc-test.git  #git仓库地址

    # rabbitmq的配置
    rabbitmq:
        host: 127.0.0.1
        port: 5672
        username: guest
        password: guest

# 使用bus总线刷新配置文件
management:
    endpoints:
        web:
            exposure:
                include: bus-refresh      #暴露bus-refresh节点，通过此节点刷新配置

#eureka配置
#...
```

> 1. rabbitmq的配置和bus总线配置是新添加的
> 2. rabbitmq的通信端口是5672，而后台管理端口是15672

2.3.微服务端配置

1. 修改 provider_server_11000工程和provider_server_11001工程，添加依赖（与config server是一样的）

```
<!-- 消息总线依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-bus</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
</dependency>

<!-- 监听依赖 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. 修改Controller组件

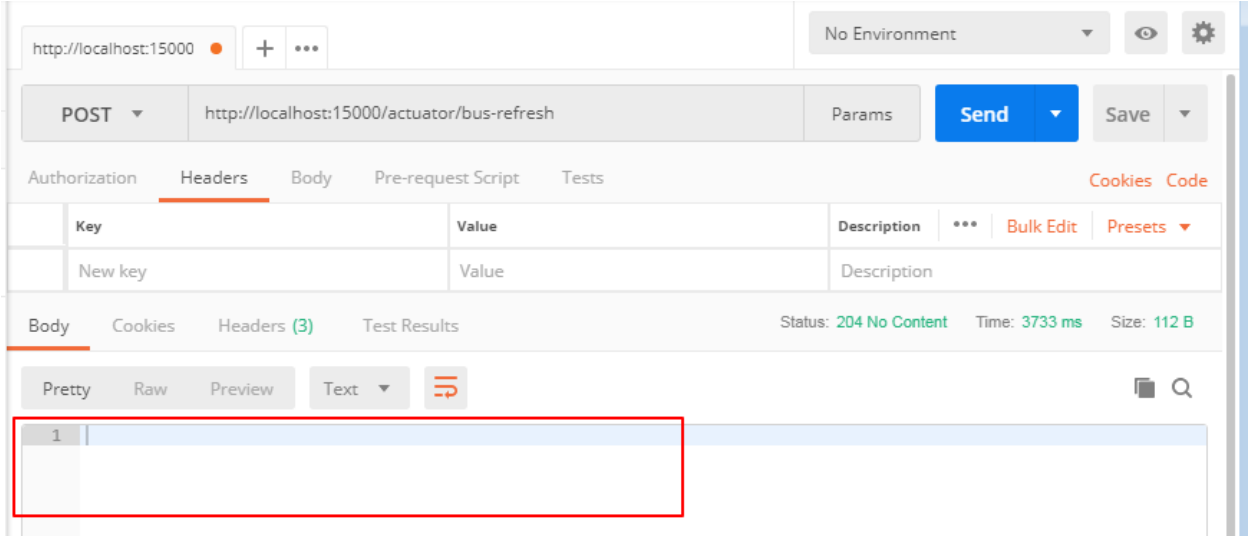
```
@RestController
@RequestMapping("/user")
@RefreshScope        //开启动态刷新
public class UserController {

    //获取配置信息中的name属性值
    @Value("${msg}")
    private String msg;

    @GetMapping("/getUserById/{userId}")
    public CommonResult<User> getUserById(@PathVariable("userId") Integer userId){
        //模拟返回业务数据
        return new CommonResult(200,"success(11000)(msg:"+msg+")",
                                    new User(userId,"张三","123"));
    }
}
```

> 在类上添加 @RefreshScope注解，开启动态刷新

2.4.测试

- 修改 Git仓库中配置文件内容（修改name属性值）
- 在Postman中发送请求：http://localhost:15000/actuator/bus-refresh （必须是post请求）

  发送post请求后，如果postman响应为空，就表示成功。



- 访问：http://localhost:14000/cart/getUserById/1?token=1 可以看到配置信息已经刷新